

Semester Project

1. Project Overview

This semester project is designed to simulate a real-world **software house environment**. Students will work in teams and follow a complete **Software Development Life Cycle (SDLC)** including requirement gathering, design, development, testing, and project management.

Each team will independently select a project of their choice (e.g., web application, mobile app, management system, etc.), and execute it using industry practices and tools.

2. Team Formation

- Students must form **groups of 4–5 members**
- Each group must assign **clearly defined roles**

Mandatory Roles (Minimum 5)

Each team must include the following roles:

1. **Project Manager (PM)**
 - Responsible for planning, coordination, and timeline management
 - Ensures tasks are assigned and completed on time
2. **Business Analyst (BA) / Requirement Engineer**
 - Responsible for requirement gathering and documentation
 - Prepares SRS document
3. **UI/UX Designer (optional but recommended)**
 - Designs wireframes, mockups, and user interface
4. **Developer(s)**
 - Responsible for coding and implementation
 - At least 1–2 members
5. **QA Engineer / Tester**
 - Responsible for testing, bug reporting, and quality assurance
 - Prepares test plan and executes testing

Note: One student may handle more than one role only if the team size is 4.

3. Tools and Platforms (Mandatory Usage)

Each team must use the following tools:

- **JIRA** – for issue tracking, sprint planning, bug reporting
- OR **Trello** – for task management (if JIRA is not used)
- **GitHub** – for code version control
- Documentation tools (MS Word / Google Docs, overleaf)

All communication, task assignment, and progress tracking must be visible on the selected platform.

4. Project Execution Phases

The project must be completed in the following structured phases:

Phase 1: Project Selection & Proposal

- Select a project idea
- Define:
 - Problem statement
 - Objectives
 - Target users
- Submit a **Project Proposal Document**

Phase 2: Requirement Gathering & Analysis

- Identify:
 - Functional Requirements
 - Non-Functional Requirements
- Create:
 - Use Case Diagrams
 - User Stories (to be added in JIRA/Trello)

Deliverable:

- Complete **Software Requirements Specification (SRS)** document

Phase 3: Project Planning

- Create:
 - Project timeline (Gantt Chart or Sprint Plan)
 - Task breakdown (Work Breakdown Structure)
- Assign tasks in **JIRA/Trello**
- Define:
 - Milestones

- Deadlines

Deliverable:

- Project Plan Document
- Active JIRA/Trello Board

Phase 4: System Design

- Create:
 - System Architecture Diagram
 - Database Design (ERD)
 - UI Wireframes / Mockups

Deliverable:

- Design Document

Phase 5: Development

- Implement the system in modules
- Follow proper coding practices
- Maintain code in GitHub with commits history

Important Requirements:

- Tasks must be linked with JIRA/Trello
- Each feature should correspond to a ticket

Phase 6: Testing & Quality Assurance

6.1 Test Planning

- Create a **Test Plan Document** including:
 - Testing scope
 - Testing types
 - Test strategy
 - Tools used

6.2 Test Case Design

- Write:

- Test Cases
- Test Scenarios

6.3 Test Execution

- Perform:
 - Functional Testing
 - Black Box Testing
 - Basic White Box concepts (if applicable)

6.4 Bug Reporting

- Report bugs on **JIRA**
 - Include:
 - Steps to reproduce
 - Expected vs actual result
 - Severity/Priority

Phase 7: Bug Fixing & Tracking

- Developers will:
 - Fix reported bugs
 - Update status on JIRA
- QA will:
 - Retest resolved issues
 - Close or reopen tickets

Deliverable:

- Bug Report Log (from JIRA/Trello)

Phase 8: Automation Testing (Basic Level)

- Perform basic automation using any tool (optional tools):
 - Selenium / Cypress / any scripting
- Automate at least:
 - 2–3 test cases

Deliverable:

- Automation scripts + brief explanation

Phase 9: Final Deployment & Demonstration

- Deploy the project (if possible)
- Prepare:
 - Final Presentation
 - System Demonstration

5. Required Deliverables

Each group must submit:

1. Project Proposal
2. SRS Document
3. Project Plan
4. Design Document
5. Source Code (GitHub link)
6. Test Plan Document
7. Test Cases & Test Results
8. Bug Reports (JIRA/Trello)
9. Automation Scripts
10. Final Presentation

6. Evaluation Criteria

Evaluation will be based on:

- Proper role distribution and teamwork
- Effective use of JIRA/Trello
- Quality of documentation (SRS, Test Plan)
- Implementation completeness
- Testing quality and bug tracking
- Use of automation
- Final demonstration

7. Important Instructions

- All team members must actively participate
- Work must be properly divided and traceable
- No copied projects are allowed
- Regular progress monitoring will be conducted
- JIRA/Trello activity will be evaluated as part of grading

8. Objective of the Project

The main goal is to ensure students:

- Understand real-world software development workflows
 - Learn collaboration and role-based responsibilities
 - Gain hands-on experience with industry tools
 - Apply testing, documentation, and project management practices
-

9. Evaluation Rubrics (Total: 100 Marks)

1. Team Roles & Collaboration (10 Marks)

Criteria	Marks	Description
Role Assignment	3	Clear definition of roles (PM, BA, Dev, QA, etc.)
Participation	4	Equal contribution from all members
Coordination	3	Effective communication and teamwork

2. Project Proposal (5 Marks)

Criteria	Marks	Description
Problem Definition	2	Clear and relevant problem statement
Objectives & Scope	3	Well-defined goals and feasibility

3. SRS Document (15 Marks)

Criteria	Marks	Description
Functional Requirements	5	Complete and clearly written
Non-Functional Requirements	3	Performance, usability, security, etc.
Use Cases / User Stories	4	Properly defined and structured
Documentation Quality	3	Clarity, formatting, completeness

4. Project Planning & Management (10 Marks)

Criteria	Marks	Description
Task Breakdown	3	Proper division of work
Timeline / Milestones	3	Realistic deadlines and planning
JIRA/Trello Usage	4	Active task tracking and updates

5. System Design (10 Marks)

Criteria	Marks	Description
Architecture Design	3	Logical and scalable design
Database Design (ERD)	3	Proper relationships and structure
UI/UX Design	4	Clear wireframes/mockups

6. Development & Implementation (15 Marks)

Criteria	Marks	Description
Functionality	6	Features implemented correctly
Code Quality	4	Clean, readable, modular code
Version Control	3	Proper use of GitHub commits
Integration	2	Modules work together properly

7. Testing & QA (15 Marks)

Criteria	Marks	Description
Test Plan	4	Well-structured and complete
Test Cases	4	Proper coverage and clarity
Testing Execution	4	Evidence of testing performed
Bug Identification	3	Ability to find relevant defects

8. Bug Tracking & Resolution (10 Marks)

Criteria	Marks	Description
Bug Reporting (JIRA)	4	Clear, detailed bug reports
Bug Resolution	4	Effective fixing of issues
Retesting	2	Proper validation of fixes

9. Automation Testing (5 Marks)

Criteria	Marks	Description
Automation Implementation	3	At least 2–3 test cases automated
Script Quality	2	Working and understandable scripts

10. Final Presentation & Demonstration (5 Marks)

Criteria	Marks	Description
Presentation Skills	2	Clear explanation of project
System Demo	3	Working and properly demonstrated system